

LLM

1) LLM

LLM (Large Language Model) is a machine-learning model trained on a very large amount of text data to learn patterns in language and generate text that resembles human language.

Large=> Trained using a very large amount of data and has a very large number of parameters

Language=> It works with human language like: English,Urdu,etc

Model=> Mathematical system that has learned patterns from data.

How LLM Works?

- **Step 1: Tokenization** => The computer chops text into smaller pieces (tokens) and immediately assigns a unique number to each piece.
- **Step 2: Looking at Meaning (Embeddings)** => The network places these numbers onto a mathematical map to understand how words relate to each other.
- **Step 3: Neural Network Processing** => The "brain" of the AI analyzes the numbers, looks at the context of your whole sentence, and predicts the numbers for the best next words.
- **Step 4: De-tokenization** => The computer converts those predicted numbers back into human words.
- **Step 5: Output** => The model displays the final text answer to you.

At a simplified level, it receives tokens and calculates probability for what tokens should come next.

2) LLM API

- a. An API allows applications to communicate with LLM. Instead of building or running a massive AI on our own computer, we send a text request (a prompt) through the internet to a powerful server, and the server instantly sends the AI's reply to our application.
- b. **E.g.: Suppose** building a MERN Application. React send some commands like “Explain JWT” to the backend. Backend sends the response to an LLM API. The LLM generated a response. Backend sends the response back to react.
React->Backend->LLM API-> LLM -> LLM API-> Backend-> React
- c. **How does the LLM API work?**
 - i. User sends a message-> Frontend sends the message to our backend-> Backend sends the request to the LLM API-> LLM provider processes the request-> LLM generates a response-> LLM API returns the response to our backend-> Backend sends the response to Frontend-> Frontend displays the response to the user
- d. **What does an API request contain?**
 - i. An LLM API commonly contains things such as model, message/input (role, content), Instructions, generation setting, tools (if supported)

- ii. What is the message role? In many LLM APIs, messages have different roles. E.g.:
 - 1. System=> Instructions for how the assistant should behave
 - 2. User=> user's message
 - 3. Assistant=> The model's previous response.

e. How to use the LLM API?

- i. To use an LLM API, we generally need:
 - 1. An account with an LLM provider.
 - 2. An API key.
 - 3. An LLM model.
 - 4. A backend application such as Node.js/Express.
 - 5. Code that sends a request to the LLM API.
 - a. For example, in a Node.js/Express backend, we can use an LLM provider's SDK:

```
import OpenAI from "openai";
const client = new OpenAI ({
  apiKey: process.env.OPENAI_API_KEY,
});
```
 - b. The API key should be stored in an environment variable:
OPENAI_API_KEY=api_key
 - c. Then we can send a request to the LLM:

```
const response = await client.responses.create({
  model: "model",
  input: "Explain JWT in simple English.",
});
console.log(response.output_text);
```
 - d. The basic idea is:
Input/Prompt-> LLM API-> LLM processes the input-> LLM generates response-> API returns response

3) Context

- a. Context is the information provided to the model as part of the current input/request that the model can use when generating its response.
- b. E.g.: user=> My name is, Assistant=> Nice to meet you, _, user=> What is my name?
If the previous conversation is included in the models input, the model can answer.
- c. Context is not necessarily permanent in memory: Application can send the previous messages as context with new requests. Application provides relevant previous info.
- d. The application manages the context when the conversation becomes too large. It may remove, summarize, or retrieve relevant information.
- e. **What goes into the context?**

- i. The application constructs the model's input/context from relevant information such as the current prompt, conversation history, system instructions, retrieved documents, and tool results.
- f. **How it works?**
 - i. **Reading everything together:** Every time send a message, AI doesn't just read a new sentence. It re-reads entire active context-window; all the past messages and instructions combine.
 - ii. **Finding Connections:** It uses the math trick called **attention** to see how words are related to each other to access everything written in that context. E.g. if name mentioned 10 minutes ago, the attention mechanism links that old name token to the current question.
 - iii. **Predicting the next word:** Based on all the words currently sitting inside its memory box, it guesses and writes the next best word, one token at a time.
- g. For a long conversation, the application may need to manage what info is included.
- h. This brings us tokens and context window.

4) Token

- a. A token is a piece of text processed by the model.
- b. e.g. Hello-> might be one token, unbelievable-> might be three tokens (un, believ, able)
- c. The exact tokenization depends on the model/tokenizer.
- d. **Used for:** Training models, processing input, vocabulary optimization.
- e. **How does the token generation work?**
 - i. **Step 1:** Tokenizer chops sentences into tokens.
 - ii. **Step 2:** AI matches every token to a specific ID, no form a giant list it learned during the list.
 - iii. **Step 3:** AI reads a list of numbers and calculates the statistical adds for what number should come next. The model produces probabilities for possible next tokens. A decoding strategy selects a token, which is then added to the output. The process repeats until the response is complete, converts the generated token IDs back into text, and adds it to the screen. It repeats this process until the full answer is written.

5) Context window

- a. A context window is the maximum number of tokens that a model can process as input/context for a request.
- b. If everything becomes too large, app may need to:
 - i. Remove old message
 - ii. Summarize old messages.
 - iii. Retrieve only relevant info.
 - iv. Split documents.

6) Chunking

- a. Break large piece of text into smaller pieces. A chunk could contain a certain number of tokens or characters.
- b. **Used for:** Search system, Question answers, knowledge retrieve.
- c. **Why do we need to chunk?**
 - i. Large documents may be too large to process efficiently.
 - ii. Smaller chunks make retrieval more precise.
 - iii. Relevant pieces can be sent to the LLM instead of the entire document.
 - iv. Embeddings represent smaller, more focused pieces of information.
- d. **How do chunks work?**
 - i. **Splitting the text**-> A long file cut into smaller paragraphs or sentences.
 - ii. **Saving the pieces**-> Each small piece is converted into computer code and saved in special db.
 - iii. **Finding the match**-> when asking a question, the system searches the db to find only the chunks that match your sentences.
 - iv. **Answering**-> These specific chunks are handed to the LLM so it can write an accurately informed answer.
- e. **What is chunk overlap?**
 - i. The amount of text that is shared between two neighboring chunks of a larger document when it is broken down, and AI reads.
 - ii. If we cut the text strictly at a certain amount of word count, we might split a sentence in half, causing the AI to lose the context of the statement. To prevent this, the system reuses a small portion of text from the end of one chunk at the beginning of the next chunk.

7) How do chunking and hallucination relate to each other?

- a. **Chunking** splits large documents into meaningful, retrievable sections, while **Tokenization** converts text into small sub-words or number that a language model process.
- b. **How do they work together?**
 - i. **Dependency**-> A single chunk usually contains many tokens. When we set a chunk size, we often measure it by token count.
 - ii. **Pipeline Flow**-> Documents are divided into chunks, and then those chunks break down into tokens and turn into vector embeddings for DB storage.

8) Hallucination

- a. Hallucination occurs when an AI generates information that is false, fabricated, or unsupported by the available evidence/context, while presenting it as if it were valid.
- b. **Why does hallucination happen?**
 - i. **Pattern Matching**-> AI predicts the next likely word instead of checking the facts.
 - ii. **Missing Knowledge**-> if AI doesn't know the true answer, it tries to fill the blank by guessing what sounds right.
 - iii. **Bad Data**-> Internet has mistakes, which the AI learns during training.
 - iv. **Outdated Knowledge.**

c. How do we reduce hallucination?

- i. **Give Real Facts First (RAG)**-> Connect the AI to a safe search tool or document library. Instead of guessing from its memory, the AI reads the real documents you hand it and answers using only those facts.
- ii. **Fine-Tuning**-> Training the AI on clean, professional data rather than random websites

9) Guardrails

- a. Safety and control system that sits between user and AI model to ensure input and output stay within safe, accurate and defined boundaries.
- b. **What can guardrails do?**
 - i. **Input validation:** prevent certain unaware inputs reaching the model.
 - ii. **Output Validation:** Check the model response before returning it.
 - iii. **Topic restrictions**
 - iv. **Format Restrictions**
 - v. **Data protection**-> Prevent sensitive data from being exposed.